

Anatomía de un agente de programación

Guía de los conceptos de *harness engineering*, explicados a partir del código de `mini-agent`: qué es un harness, qué es una herramienta, cómo funciona el bucle del agente y por qué cada decisión está tomada así.

Interacso · octubre de 2026

Referencia: repositorio `~/desarrollo/mini-agent` (TypeScript, Bun, `@anthropic-ai/sdk`)

Índice

1. La idea en una página	3
2. ¿Qué es un harness?	3
3. Cómo se habla con el modelo: la API Messages	4
4. ¿Qué es una herramienta (tool)?	5
5. El bucle del agente	7
6. Un turno completo, paso a paso	8
7. Streaming, thinking y effort	9
8. Caché de prompt y coste	9
9. Seguridad: permisos y sandbox	10
10. Errores: tres categorías, tres tratamientos	11
11. Herramientas en paralelo	12
12. Límites y gestión del contexto	12
13. Limitaciones conocidas del proyecto	13
14. Mapa del código	14
15. Glosario	14

1. La idea en una página

Un modelo de lenguaje, por sí solo, **no hace nada**: recibe texto y devuelve texto. No lee ficheros, no ejecuta comandos y no recuerda la conversación anterior. Cada llamada a la API es independiente.

Un **agente** aparece cuando le pones alrededor un programa que hace tres cosas:

- Le dice al modelo qué **herramientas** existen (leer un fichero, ejecutar un comando...).
- Cuando el modelo responde «quiero usar la herramienta X con estos parámetros», el programa **la ejecuta de verdad** y le devuelve el resultado.
- Repite esto en un **bucle** hasta que el modelo dice «he terminado».

Ese programa es el **harness**. El modelo decide *qué* hacer; el harness decide *cómo*, *con qué límites* y *con qué seguridad* se hace.

Fórmula para recordar

Agente = modelo + herramientas + bucle + harness. El modelo es el «cerebro»; las herramientas son las «manos»; el bucle es el «ciclo de trabajo»; el harness es todo lo demás: permisos, límites, errores, contexto, caché, interfaz y coste.

mini-agent es un harness mínimo (~740 líneas) escrito a mano, sin frameworks, precisamente para que todas esas piezas estén a la vista. Este documento las recorre una a una.

2. ¿Qué es un harness?

Harness significa literalmente «arnés»: el equipo que se pone a un caballo para que su fuerza se convierta en trabajo útil (tirar de un carro) en lugar de en movimiento sin control. En IA el término se usa igual: **el harness es el software que rodea al modelo y canaliza su capacidad hacia una tarea concreta.**

Harness engineering es la disciplina de diseñar ese software. Dos agentes con el mismo modelo pueden rendir de forma muy distinta según el harness: qué herramientas les das, cómo describes cada una, qué haces cuando algo falla, cómo gestionas el historial, etc.

Responsabilidades de un harness

Responsabilidad	Pregunta que responde	Dónde está en mini-agent
Bucle	¿Cuándo llamo al modelo y cuándo paro?	<code>src/agent.ts</code> → <code>send()</code>
Herramientas	¿Qué puede hacer el modelo en el mundo real?	<code>src/tools/*.ts</code>
Validación	¿El modelo me ha pedido algo bien formado?	Zod en <code>runTools()</code>
Permisos	¿El usuario quiere que esto ocurra?	<code>src/permissions.ts</code>
Confinamiento	¿A qué partes del sistema puede llegar?	<code>src/sandbox.ts</code>
Errores	¿Qué hago si algo falla?	checkpoint + <code>is_error</code>
Contexto	¿Qué ve el modelo en cada llamada?	historial <code>messages</code> , <code>truncate()</code>

Responsabilidad	Pregunta que responde	Dónde está en mini-agent
Caché y coste	¿Cuánto cuesta cada vuelta?	<code>cache_control</code> , <code>/usage</code>
Interfaz	¿Qué ve y controla el humano?	<code>src/index.ts</code> , <code>src/ui.ts</code>

Idea clave

Al modelo hay que tratarlo como un colaborador muy capaz pero **no confiable**: lo que pide puede estar mal formado, ser peligroso o venir manipulado por texto que ha leído (por ejemplo, un fichero con instrucciones maliciosas, lo que se llama *prompt injection*). El harness es la capa que pone las reglas.

3. Cómo se habla con el modelo: la API Messages

Antes de entender el bucle hay que entender el formato de una llamada. La API de Anthropic (*Messages API*) recibe una petición con estas piezas principales:

Campo	Qué es
<code>model</code>	Qué modelo usar, p. ej. <code>claude-opus-5-5</code> .
<code>system</code>	Instrucciones generales (el <i>system prompt</i>): quién es, cómo debe trabajar.
<code>tools</code>	Lista de herramientas disponibles, cada una con nombre, descripción y un JSON Schema de sus parámetros.
<code>messages</code>	Todo el historial de la conversación, alternando <code>user</code> y <code>assistant</code> .
<code>max_tokens</code>	Máximo de tokens que puede generar en esta respuesta.
<code>thinking</code>	Si el modelo puede «pensar» antes de responder (razonamiento visible o resumido).

La API no tiene memoria

Este es el punto más importante y el menos intuitivo: **el servidor no recuerda nada entre llamadas**. Si en la vuelta 5 quieres que el modelo sepa lo que pasó en las vueltas 1-4, tienes que reenviarle todo ese historial. Por eso el agente guarda un array `messages` que crece con cada vuelta y lo manda entero en cada petición.

Bloques de contenido

Cada mensaje no es un simple string, sino una lista de **bloques** de distintos tipos:

Bloque	Quién lo genera	Para qué sirve
<code>text</code>	usuario o modelo	Texto normal.
<code>thinking</code>	modelo	Razonamiento previo a la respuesta. Hay que devolverlo tal cual en el historial.
<code>tool_use</code>	modelo	«Quiero llamar a esta herramienta con este input». Lleva un <code>id</code> único.
<code>tool_result</code>	harness (en un mensaje <code>user</code>)	El resultado de ejecutar la herramienta. Referencia el <code>tool_use_id</code> .

stop_reason: por qué ha parado el modelo

Cada respuesta trae un campo `stop_reason` que dice por qué el modelo dejó de generar. Es la señal que gobierna el bucle:

stop_reason	Significado	Qué hace mini-agent
end_turn	Ha terminado de responder.	Guarda la respuesta y acaba el turno.
tool_use	Quiere usar una o varias herramientas.	Las ejecuta, añade los resultados y vuelve a llamar.
max_tokens	Se ha quedado sin tokens a mitad de respuesta.	Si había un <code>tool_use</code> , no lo ejecuta (estaría cortado) y descarta el turno. Si era texto, lo guarda y avisa.
pause_turn	El servidor ha pausado un turno largo (solo con herramientas de servidor).	Reenvía para continuar.
refusal	El modelo se niega a continuar.	Descarta el turno entero.
stop_sequence	Ha encontrado una secuencia de parada configurada.	Igual que <code>end_turn</code> .
otro	Valor no previsto.	Avisa y termina (ver la sección 13 sobre un fallo aquí).

4. ¿Qué es una herramienta (tool)?

Una herramienta es **una función de tu programa que el modelo puede pedir que ejecute**. El modelo nunca la ejecuta él mismo: solo genera una petición estructurada. Quien la ejecuta es el harness, en tu máquina, con tus permisos.

Las tres partes de una herramienta

- **Nombre:** identificador, p. ej. `read_file`.
- **Descripción:** texto en lenguaje natural que el modelo lee para decidir cuándo y cómo usarla. Es, en la práctica, parte del prompt: una mala descripción produce un mal uso.
- **Input schema:** un JSON Schema que define los parámetros (`path`, `offset...`), sus tipos y cuáles son obligatorios.

El ciclo de una llamada, en JSON

1) El harness declara la herramienta en la petición:

```
{ "name": "read_file",
  "description": "Lee un fichero de texto del proyecto. Devuelve las líneas numeradas...",
  "input_schema": {
    "type": "object",
    "properties": { "path": { "type": "string" }, "offset": { "type": "integer" } },
    "required": ["path"] } }
```

2) El modelo responde con `stop_reason: "tool_use"` y un bloque así:

```
{ "type": "tool_use", "id": "toolu_01A...", "name": "read_file",
  "input": { "path": "src/sandbox.ts" } }
```

3) El harness lee el fichero y en la siguiente petición añade un mensaje **user** con:

```
{ "type": "tool_result", "tool_use_id": "toolu_01A...",
  "content": "1\timport { realpathSync } from \"node:fs\";\n2\t..." }
```

Si algo falla, el mismo bloque lleva **"is_error": true** y el mensaje de error como contenido. El modelo lo lee y normalmente se corrige solo (por ejemplo, vuelve a leer el fichero antes de intentar otra edición).

Cómo se define una herramienta en mini-agent

El contrato está en **src/tools/tool.ts**. Cada herramienta es un objeto **AgentTool**:

```
export interface AgentTool<S extends z.ZodObject = z.ZodObject> {
  name: string;
  description: string;
  schema: S; // schema de Zod
  readOnly: boolean; // las de solo lectura no piden permiso
  summarize(input): string; // texto corto para la UI y el permiso
  run(input, ctx: ToolContext): Promise<string>;
}
```

src/tools/tool.ts

La decisión interesante es usar **Zod como única fuente de verdad**: del mismo schema sale el JSON Schema que ve el modelo (**z.toJSONSchema()**) y la validación que se hace antes de ejecutar (**schema.safeParse()**). Así es imposible que la descripción y la validación se desincronicen.

Las cinco herramientas

Herramienta	Qué hace	readOnly	Detalles de diseño
read_file	Lee un fichero con líneas numeradas.	sí	Paginación con offset/limit ; avisa de cuántas líneas quedan.
write_file	Crea o sobrescribe un fichero entero.	no	Crea directorios intermedios.
edit_file	Sustituye un fragmento exacto.	no	Exige que old_string sea único, salvo replace_all .
grep	Busca con regex.	sí	Usa ripgrep si existe; máximo 200 líneas.
bash	Ejecuta un comando.	no	Timeout, salida truncada, sin aislamiento .

¿Por qué edit_file exige un texto único?

Porque es la forma más robusta de que el modelo edite sin ambigüedad. Si **old_string** aparece dos veces, el harness no adivina: devuelve un error pidiendo más contexto. Es un patrón general: **cuando el input es ambiguo, mejor un error claro que una acción equivocada**.

Los mensajes de error son prompts

Fíjate en que los errores de las herramientas dicen qué hacer a continuación: «old_string no aparece en el fichero. Vuelve a leerlo y copia el texto exacto». El destinatario del error es el modelo, así que el error debe ayudarle a corregirse.

5. El bucle del agente

El bucle (*agent loop*) es el corazón del harness. Su forma general es siempre la misma:

```
mensajes.push({ role: "user", content: textoDelUsuario })
repetir hasta maxTurns:
  respuesta = llamarAlModelo(system, tools, mensajes)
  según respuesta.stop_reason:
    "end_turn": mensajes.push(respuesta); terminar
    "tool_use": mensajes.push(respuesta)
                  resultados = ejecutar(cada tool_use de la respuesta)
                  mensajes.push({ role: "user", content: resultados })
                  seguir          # otra vuelta
    otro: manejar el caso y terminar
```

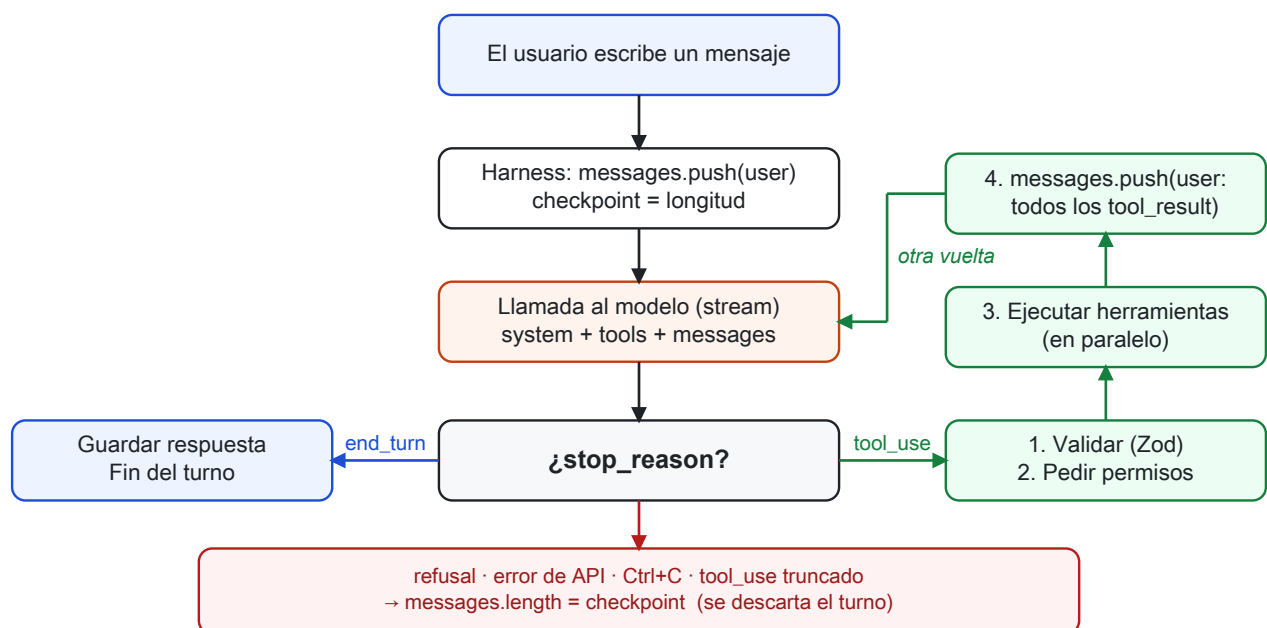


Figura 1. El bucle de mini-agente. Cada vuelta verde es una llamada más al modelo.

Vocabulario: turno y vuelta

- **Turno**: todo lo que pasa desde que el usuario escribe un mensaje hasta que el agente termina de responder. Es una llamada a `agent.send()`.
- **Vuelta** (*turn* en el código, a veces *step* o *iteration*): una llamada al modelo dentro de ese turno. Un turno puede tener muchas vueltas: leer, buscar, editar, ejecutar tests, corregir...
- **maxTurns** (50 por defecto) limita las vueltas por turno, para que un agente atascado no consuma tokens indefinidamente.

El bucle real

```
async send(userText: string, signal: AbortSignal) {
  const checkpoint = this.messages.length;
  this.messages.push({ role: "user", content: userText });
  try {
    for (let turn = 0; turn < this.opts.maxTurns; turn++) {
      const message = await this.callModel(signal);
      this.trackUsage(message);
      switch (message.stop_reason) {
        case "end_turn": this.messages.push(...); return;
        case "tool_use": {
          this.messages.push({ role: "assistant", content: message.content });
          const results = await this.runTools(toolUses, signal);
          this.messages.push({ role: "user", content: results });
          continue;
        }
        // refusal, max_tokens, pause_turn, default...
      }
    }
  } catch (err) {
    this.messages.length = checkpoint; // descartar el turno
    throw err;
  }
}
```

src/agent.ts (simplificado)

6. Un turno completo, paso a paso

Supón que el usuario escribe: «*el test de sandbox falla, arréglalo*». Esto es lo que ocurre por dentro:

#	Quién	Qué pasa	Historial tras el paso
1	Harness	Guarda checkpoint = 0 y añade el mensaje del usuario.	[user]
2	Modelo	Piensa y pide bash: bun test . stop_reason = tool_use.	[user, assistant]
3	Harness	Valida el input, pregunta permiso («¿Permitir bash: bun test?»), el usuario dice «s», ejecuta.	
4	Harness	Añade un mensaje user con el tool_result (salida del test fallido).	[..., user]
5	Modelo	Lee el error y pide read_file src/sandbox.ts y read_file test/tools.test.ts a la vez.	[..., assistant]
6	Harness	Son de solo lectura: sin preguntar, las ejecuta en paralelo. Añade los dos resultados en un único mensaje.	[..., user]
7	Modelo	Pide edit_file con el cambio.	[..., assistant]
8	Harness	Pide permiso, ejecuta y devuelve «Editado src/sandbox.ts».	[..., user]
9	Modelo	Vuelve a ejecutar bun test ; ahora pasa.	...
10	Modelo	Escribe un resumen. stop_reason = end_turn.	[..., assistant]

Han sido **un turno y cinco vueltas** (cinco llamadas a la API). En cada llamada se envía el historial completo, que va creciendo. Esto tiene una consecuencia directa en coste y latencia, que es lo que

resuelve la caché (sección 8).

7. Streaming, thinking y effort

Streaming

En lugar de esperar a la respuesta completa, `client.beta.messages.stream()` recibe la respuesta como una secuencia de eventos según se genera: empieza un bloque, llega un trozo de texto, termina un bloque... mini-agent los pinta en la terminal en tiempo real (`renderStream()`) y, al final, `stream.finalMessage()` devuelve el mensaje completo para el bucle.

eager_input_streaming y validación propia

Normalmente el servidor espera a tener el JSON completo de un `tool_use` antes de enviarlo. Con `eager_input_streaming: true` (en `toApiTool()`) los inputs llegan a trozos según se generan, lo que va mejor cuando el input es un fichero entero. El precio es que **el servidor ya no garantiza que el input cumpla el schema**. Por eso mini-agent valida con Zod antes de ejecutar, y si el JSON llega roto reintenta la llamada hasta 2 veces (`MAX_JSON_RETRIES`).

Thinking adaptativo

Con `thinking: { type: "adaptive" }` el modelo decide por sí mismo cuánto razonar antes de responder: poco en preguntas sencillas, más en problemas difíciles. Con `display: "summarized"` se recibe un resumen de ese razonamiento, que mini-agent muestra en gris, antes de la respuesta.

Regla importante: los bloques `thinking` forman parte de la respuesta del asistente y **se guardan en el historial tal cual**. Por eso el código hace `messages.push({ role: "assistant", content: message.content })` con el contenido completo, sin filtrar.

Effort

`output_config: { effort }` ajusta cuánto esfuerzo dedica el modelo (de `low` a `max`). Más esfuerzo suele significar respuestas más cuidadosas, pero también más tokens y más latencia. mini-agent usa `high` por defecto y se puede cambiar con `--effort`.

8. Caché de prompt y coste

Como en cada vuelta se reenvía el historial completo, sin caché el coste crecería muy rápido: en la vuelta 20 estarías pagando 20 veces por el system prompt y por todo lo leído hasta entonces.

La **caché de prompt** resuelve esto: si el principio de la petición (el **prefijo**) es idéntico byte a byte al de una petición reciente, el servidor reutiliza el trabajo ya hecho. Esos tokens se cobran como *cache read*, mucho más baratos que el input normal, y la respuesta llega antes.

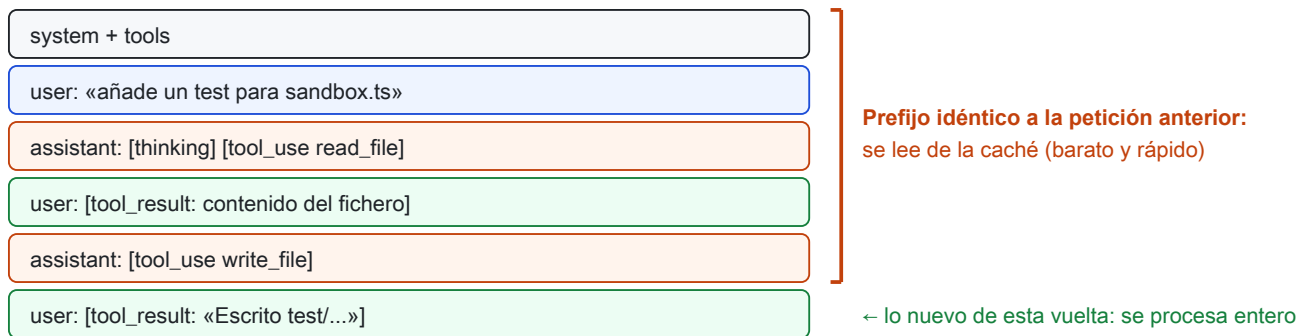


Figura 2. En cada vuelta solo es «nuevo» el final del historial; todo lo anterior es un prefijo cacheable.

Qué hace mini-agent para aprovecharla

- **System prompt estable:** no incluye la fecha ni nada que cambie entre peticiones (ver el comentario en `buildSystemPrompt()`).
- **Orden fijo de herramientas:** las herramientas forman parte del prefijo; cambiar su orden invalidaría la caché (`src/tools/index.ts`).
- `cache_control: { type: "ephemeral" }` a nivel de petición: marca automáticamente el último bloque cacheable, así cada vuelta reutiliza todo lo anterior.
- **Historial append-only:** nunca se edita un mensaje antiguo, solo se añade al final o se recorta el final. Editar el mensaje 3 de 40 invalidaría la caché desde ese punto.

Medir el coste: `/usage`

Cada respuesta trae `usage` con cuatro contadores, que `trackUsage()` acumula: tokens de input normal, `cache read`, `cache write` (lo que se guarda en caché por primera vez) y output. `/usage` los muestra con una estimación en dólares. Si `cache read` es mucho mayor que `input` en una sesión larga, la caché está funcionando.

Atención

La tabla de precios de `src/index.ts` está escrita a mano. Antes de fiarte de la cifra en dólares, compárala con la página oficial de precios de Anthropic.

9. Seguridad: permisos y sandbox

Todo lo que viene del modelo es **input no confiable**. No porque el modelo sea malicioso, sino porque puede equivocarse o haber leído texto diseñado para manipularlo. El harness pone dos barreras independientes.

Barrera 1: permisos (`src/permissions.ts`)

Antes de ejecutar algo con efectos, se pregunta al humano:

Caso	Comportamiento
Herramienta <code>readOnly</code> (<code>read_file</code> , <code>grep</code>)	Se ejecuta sin preguntar.
Modo <code>ask</code> (por defecto)	Pregunta «¿Permitir bash: bun test? [s]í / [n]o / [a]lways».
Respuesta <code>a</code>	No vuelve a preguntar por esa herramienta en toda la sesión.
Modo <code>yolo</code> (<code>--yes</code>)	Todo permitido sin preguntar.
Respuesta <code>n</code>	No se ejecuta; el modelo recibe «El usuario ha denegado esta acción. Pregúntale cómo quiere continuar».

Fíjate en que la denegación **no rompe el bucle**: se convierte en un `tool_result` con error. El system prompt además indica al modelo que no reintente lo denegado.

Barrera 2: sandbox de rutas (src/sandbox.ts)

Las herramientas de ficheros pasan cada ruta por `resolveInRoot(root, ruta)`, que garantiza que el fichero final está dentro del directorio del proyecto. Hace dos comprobaciones:

- **Léxica**: resuelve `..` y rutas absolutas. `../../etc/passwd` o `/etc/passwd` se rechazan.
- **Real**: sigue los symlinks del ancestro más cercano que exista. Un enlace `escape -> /tmp` dentro del proyecto pasaría la comprobación léxica, pero no esta.

La limitación importante

bash no pasa por el sandbox. Un comando puede leer, borrar o enviar por red cualquier cosa a la que tú tengas acceso. El confinamiento solo protege las herramientas de ficheros. Por eso el modo por defecto pregunta antes de cada comando, y por eso `--yes` es peligroso. Un sandbox real para bash requiere `sandbox-exec` en macOS, un contenedor o una máquina virtual.

10. Errores: tres categorías, tres tratamientos

Una de las decisiones más importantes de un harness es qué hacer cuando algo falla. mini-agent distingue tres casos:

Tipo	Ejemplos	Tratamiento	Por qué
Error de herramienta	Fichero inexistente, <code>old_string</code> no encontrado, input inválido, permiso denegado.	Se devuelve al modelo como <code>tool_result</code> con <code>is_error: true</code> . El bucle sigue.	El modelo puede corregirse: es información, no un fallo del sistema.
Error de API	Credenciales, rate limit, error 500, petición mal formada.	Se descarta el turno (<code>messages.length = checkpoint</code>) y se muestra al usuario.	El modelo no puede arreglarlo; hay que avisar al humano.
Interrupción	Ctrl+C, <code>refusal</code> , <code>tool_use</code> truncado.	Se descarta el turno.	El estado es incompleto y no es seguro continuar.

El checkpoint

Al empezar cada turno se guarda `checkpoint = messages.length`. Si el turno tiene que descartarse, basta con `messages.length = checkpoint`: se elimina todo lo añadido en este turno y el historial vuelve exactamente al estado anterior. Al no tocar nada del principio, la caché y los bloques de thinking anteriores siguen siendo válidos.

Trampa: el historial no es el mundo

Recortar el historial **no deshace** lo que las herramientas ya hicieron. Si en la vuelta 3 se editó un fichero y en la vuelta 5 el usuario pulsa Ctrl+C, el fichero sigue editado pero el modelo ya no lo recuerda. Es la mejora más interesante que tiene pendiente el proyecto (sección 13).

Ctrl+C y AbortSignal

Cada turno crea un `AbortController`. Su `signal` se pasa a la llamada a la API, a cada herramienta (`bash` y `grep` matan su proceso) y a la pregunta de permisos. Al pulsar Ctrl+C se aborta todo a la vez y el turno se descarta.

11. Herramientas en paralelo

El modelo puede pedir varias herramientas en una sola respuesta (p. ej. leer tres ficheros). `runTools()` lo hace en dos fases:

- **Fase 1, secuencial:** para cada `tool_use`, busca la herramienta, valida el input con Zod y pide permiso si hace falta. Es secuencial para que no se mezclen varias preguntas en la terminal.
- **Fase 2, en paralelo:** ejecuta todo lo aprobado con `Promise.all`.
- **Un único mensaje:** todos los `tool_result` se envían juntos en un mismo mensaje `user`. Si se separaran en varios mensajes, el modelo tendería a dejar de pedir herramientas en paralelo.

Matiz

Paralelizar lecturas es seguro. Paralelizar **escrituras** no: dos `edit_file` sobre el mismo fichero pueden leer el original a la vez y la segunda escritura borra la primera. Lo habitual es paralelizar solo las herramientas `readOnly`.

12. Límites y gestión del contexto

El modelo tiene una **ventana de contexto** finita: un máximo de tokens entre system prompt, herramientas e historial. Todo lo que el harness mete en el historial ocupa sitio. Hay varios límites para controlarlo:

Límite	Valor	Qué protege
<code>maxTurns</code>	50 vueltas por turno	Que un agente atascado gaste sin fin.
<code>max_tokens</code>	64 000 por respuesta	El tamaño de cada respuesta del modelo.

Límite	Valor	Qué protege
<code>truncate()</code>	30 000 caracteres	Que una salida enorme (un log, un <code>cat</code>) llene el contexto. Conserva el principio y el final.
<code>read_file</code>	2 000 líneas por defecto	Ficheros grandes: el modelo pagina con <code>offset</code> .
<code>grep</code>	200 coincidencias	Búsquedas demasiado amplias: pide afinar.
<code>bash</code>	timeout 120 s	Comandos que no terminan.

Lo que **no** hay todavía es gestión activa del contexto: en una sesión muy larga el historial acabará superando la ventana. Las técnicas habituales son recortar `tool_result` antiguos (el contenido de un fichero leído hace 30 vueltas ya no suele hacer falta) o **compactar**: pedir al modelo un resumen de la conversación y continuar a partir de él.

Fallbacks del servidor

mini-agent activa `fallbacks: "default"` (con la beta `server-side-fallback-2026-07-01`). Según el código, si un clasificador de seguridad rechaza la petición, el propio servidor la reintenta con otro modelo, y la respuesta incluye un bloque `fallback` que el harness muestra («X rechazó; continúa Y»). Si toda la cadena rechaza, llega `stop_reason: refusal`.

13. Limitaciones conocidas del proyecto

Del análisis del código salen estos puntos. Son buenos ejercicios porque cada uno obliga a pensar una decisión de harness:

#	Problema	Consecuencia	Idea de solución
1	Al descartar un turno se pierden también las vueltas anteriores que ya tuvieron efectos.	El historial y el disco dejan de coincidir; el modelo «olvida» cambios hechos.	Volver al último par <code>tool_use/tool_result</code> completo, o informar al modelo.
2	La rama <code>default</code> del switch guarda un <code>tool_use</code> sin <code>tool_result</code> .	Todas las peticiones siguientes fallan hasta <code>/reset</code> .	Tratarlo como <code>max_tokens</code> .
3	Sin gestión de contexto.	Una sesión larga acaba bloqueada.	Recortar resultados antiguos o compactar.
4	Escrituras en paralelo.	Ediciones perdidas en el mismo fichero.	Paralelizar solo <code>readOnly</code> .
5	<code>bash</code> espera a que se cierren los pipes.	Se cuelga con procesos en segundo plano.	Matar el grupo de procesos; dejar de esperar al salir <code>bash</code> .
6	<code>a</code> aprueba toda la herramienta; Enter vacío = sí.	Un «always» en <code>bash</code> lo permite todo.	Reglas por patrón, p. ej. <code>bash(git status*)</code> ; default «no».
7	El bucle no tiene tests.	Los fallos 1-2 no se detectan.	Tests con un cliente falso que devuelva respuestas fijadas.

14. Mapa del código

Fichero	Líneas	Conceptos que contiene
<code>src/index.ts</code>	105	CLI (<code>parseArgs</code>), REPL, comandos <code>/reset /usage /exit</code> , Ctrl+C, coste.
<code>src/agent.ts</code>	233	Bucle, <code>stop_reason</code> , checkpoint, streaming, permisos + ejecución, uso de tokens, system prompt.
<code>src/permissions.ts</code>	35	Modos <code>ask/yolo</code> , <code>readOnly</code> , «always».
<code>src/sandbox.ts</code>	41	Confinamiento de rutas, symlinks.
<code>src/tools/tool.ts</code>	51	Contrato <code>AgentTool</code> , Zod → JSON Schema, <code>ToolError</code> , <code>truncate</code> .
<code>src/tools/*.ts</code>	~180	Las cinco herramientas.
<code>test/tools.test.ts</code>	72	Tests de herramientas y sandbox.

Orden de lectura recomendado

1. `src/tools/tool.ts`: qué es una herramienta.
2. `src/tools/read-file.ts` y `edit-file.ts`: dos herramientas reales.
3. `src/agent.ts`, función `send()`: el bucle.
4. `runTools()`: validación, permisos y paralelismo.
5. `src/permissions.ts` y `src/sandbox.ts`: seguridad.
6. `src/index.ts`: cómo se conecta todo con la terminal.

15. Glosario

Término	Definición
Agente	Programa en el que un modelo decide qué acciones tomar y un bucle las ejecuta hasta completar una tarea.
Harness	Todo el software alrededor del modelo: bucle, herramientas, permisos, límites, errores, contexto, interfaz.
Herramienta (tool)	Función del harness que el modelo puede pedir ejecutar, descrita por nombre, descripción y JSON Schema.
<code>tool_use</code>	Bloque en el que el modelo pide ejecutar una herramienta, con un <code>id</code> y un <code>input</code> .
<code>tool_result</code>	Bloque con el que el harness devuelve el resultado, referenciando el <code>tool_use_id</code> .
Bucle de agente	Ciclo llamar al modelo → ejecutar herramientas → devolver resultados → repetir.
Turno	Un mensaje del usuario y todo el trabajo del agente hasta responder.
Vuelta	Una llamada al modelo dentro de un turno.
<code>stop_reason</code>	Motivo por el que el modelo dejó de generar; gobierna el bucle.
System prompt	Instrucciones generales que acompañan a cada petición.
Ventana de contexto	Máximo de tokens que el modelo puede considerar en una llamada.

Término	Definición
Token	Unidad en la que el modelo procesa y se factura el texto (aprox. un trozo de palabra).
Caché de prompt	Reutilización en el servidor de un prefijo idéntico de la petición; abarata y acelera.
Append-only	Política de solo añadir al final del historial, nunca editar lo anterior.
Checkpoint	Longitud del historial al empezar un turno; permite descartarlo.
Streaming	Recibir la respuesta como eventos según se genera.
Thinking adaptativo	El modelo decide cuánto razonar antes de responder.
Effort	Parámetro que regula cuánto esfuerzo (y tokens) dedica el modelo.
Sandbox	Mecanismo que limita a qué recursos puede acceder una acción.
Prompt injection	Texto que el modelo lee (en un fichero, una web...) y que intenta darle instrucciones.
Zod	Librería de TypeScript para definir schemas y validar datos en tiempo de ejecución.
JSON Schema	Estándar para describir la forma de un objeto JSON; es lo que la API usa para los inputs.